

Appointment Booking System

Goal

Build a full-stack app where an **Admin** publishes availability slots and a **Customer** books/cancels them. The system must remain correct under **real-world conditions** (concurrency, retries, auth boundaries, time rules).

Roles

Admin

- Create availability slots
- View all slots and associated bookings
- Cancel a slot (define behavior for booked slots; document it)

Customer

- View available slots
 - Book a slot
 - Cancel **their own** booking
 - View "My bookings"
-

Core Requirements

Authentication & Authorization

- Login required for booking and all admin actions
 - Role-based access enforced on the backend:
 - Admin-only: create/cancel slots, view all bookings
 - Customer-only: book/cancel own booking, view own bookings
 - No "security by UI hiding"—server must reject forbidden actions
-

Data & Business Rules

Slot

Minimum fields:

- id
- start_time (ISO timestamp)
- end_time (ISO timestamp)
- status (AVAILABLE, BOOKED, CANCELLED)
- created_by (admin user id)
- created_at

Validation:

- start_time < end_time
- Minimum duration (choose e.g. 10/15 mins; document)
- **No overlapping slots** for the same resource (assume a single doctor/staff is fine)
- Timezone handling must be consistent (recommended: store UTC in DB)

Booking

Minimum fields:

- id
- slot_id
- user_id
- status (BOOKED, CANCELLED)
- created_at, cancelled_at

Rules:

- A booking belongs to exactly one slot and one customer
- Cancellation rules:

- Customer can cancel only their own booking
 - Define whether cancellation frees slot back to AVAILABLE (recommended) and document the transition logic
-

Complex Areas (Must Be Handled Correctly)

1) Double Booking / Concurrency Safety (Mandatory)

Two customers can attempt booking the same slot at the same time. Only one must succeed.

Expected implementation includes **DB-level guarantees**, such as:

- Transaction + row locking (SELECT ... FOR UPDATE), or
- Atomic update strategy (UPDATE slot SET status=BOOKED WHERE id=? AND status=AVAILABLE), or
- Unique constraint ensuring only one active booking per slot

Conflict should return a clean response (recommended: 409 Conflict).

2) Slot Overlap Rules

Admin slot creation must prevent time overlaps. Define boundary rules clearly:

- e.g. [start, end) vs [start, end] behavior
- Must be enforced on backend (UI validation alone is insufficient)

3) Cancellation Semantics (State Machine)

Define valid transitions and enforce them:

- Slot cannot be BOOKED without an active booking
- Booking cannot remain active if slot is AVAILABLE
- Avoid deleting rows; prefer status + timestamps

4) Authorization Boundaries

Prevent:

- Customer canceling other users' bookings by guessing IDs
- Customer viewing global bookings/slots if restricted
- Admin endpoints accessible without role checks

5) Idempotency / Retries

Handle:

- double clicks on Book
- request retries due to flaky networks

Return stable results (either idempotent booking or correct duplicate prevention).

6) "Available" Definition

Available slots listing must exclude:

- past slots
- cancelled slots
- booked slots

Sorting and filtering must be consistent server-side.

7) Error Handling

Use proper status codes:

- 400 validation
- 401/403 auth/role
- 404 missing resource
- 409 booking conflict

Provide useful error payloads.

Suggested API Surface (Flexible, but keep consistent)

Auth

- POST /api/auth/login
- GET /api/auth/me
- (Optional) POST /api/auth/register

Slots

- GET /api/slots?date=YYYY-MM-DD&status=AVAILABLE&sort=start&page=1
- POST /api/admin/slots
- PATCH /api/admin/slots/:id/cancel

Bookings

- POST /api/bookings body: { slot_id }
 - GET /api/bookings/me
 - PATCH /api/bookings/:id/cancel
-

UI Requirements (React Assessment Focus)

Customer: Slots Listing Page (Primary Assessment Page)

This page should demonstrate real-world React skills: URL-synced state, stable rendering, caching, optimistic updates, and clean UX.

Layout

Top Bar

- Title: "Book an Appointment"
- Selected date subtitle ("Showing availability for ...")

Sticky Filter Row

- Date picker (default = today)
- Time window chips: Morning / Afternoon / Evening

- Sort dropdown: Earliest first (default), Latest first
- Clear filters (only when active)

Slots Grid

- Cards (2–3 columns desktop, 1 column mobile)
- Each card shows:
 - time range (start–end)
 - duration
 - status badge
 - action button based on state

Right Rail / Drawer

- “My booking summary” (next upcoming booking + count for selected day)

Required Behaviors (React Signals)

- **URL-synced filters and pagination**
 - Example: ?date=2026-01-07&window=morning&sort=start&page=2
- **Pagination or infinite scroll** (pick one)
- **Optimistic booking + rollback**
 - On “Book”: show immediate pending state
 - On 409: rollback and show inline error (“Someone else booked it”)
- **Request cancellation / stable UI**
 - Avoid flicker on refetch
 - Cancel in-flight requests on rapid filter changes (AbortController / query cancellation)
- **Loading/empty/error states**
 - Skeleton loader on first load
 - Empty state: “No slots available”

- Retry action on failure

Slot Card State Handling

- AVAILABLE → "Book"
 - BOOKING_PENDING → disabled "Booking..."
 - BOOKED_BY_YOU → "Cancel"
 - BOOKED_BY_OTHER → disabled "Unavailable"
 - CANCELLED → greyed out
-

Admin UI (Minimum)

- Slots list (filter by date/status)
 - Create slot form (single slot required)
 - Cancel slot action
 - Show booking info on slot if booked
-

Deliverables (Must Be Included)

1) Git Repository

- /backend
- /frontend
- .env.example
- README.md

2) README.md must include

- Stack + versions
- Setup steps (local)
- Environment variables explained

- DB migrations instructions
- How to run tests
- Key design decisions:
 - How overlap is prevented
 - How double-booking is prevented (the exact mechanism)
 - Cancellation behavior rules
- Known limitations + improvements

3) Database + Migrations

- Real DB with schema and migrations
- Seed script optional (bonus)

4) API Documentation

Provide one:

- Postman collection export, or
- OpenAPI/Swagger spec, or
- README endpoint docs with request/response examples

5) Tests (Minimum 3 meaningful)

Must include:

- booking success
- booking conflict (409) / concurrency prevention
- authorization test (customer cannot create slots OR cannot cancel others' booking)

6) Bonus (Optional)

- Deployment (frontend + backend URLs), or
- Docker compose for local one-command run

Evaluation Criteria

- Correctness under concurrency + retries
 - Clean API + validation + error codes
 - Role enforcement & data privacy
 - React quality on listing page (URL state, caching, optimistic UI, stability)
 - Code structure, readability, tests, docs
-

Notes

You may use any stack (React/Next + Node/Django/FastAPI). The focus is correctness + thoughtful UI state handling, not pixel-perfect design.